

Virtualization

Complete Notes

for College Exams

Topper-style notes covering definitions, ASCII diagrams, comparison tables and exam questions — designed to help you score full marks.

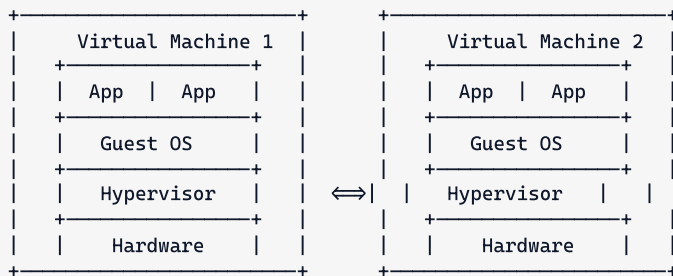


Table of Contents

Use the clickable chapter links to jump directly to any topic. Every entry is hyperlinked to its page in the PDF.

01	Introduction to Virtualization	p. 3
02	Benefits & Use Cases	p. 4
03	Types of Virtualization	p. 5
04	Hypervisor (VMM) — The Heart of Virtualization	p. 6
05	Type 1 Hypervisor (Bare-Metal)	p. 7
06	Type 2 Hypervisor (Hosted)	p. 8
07	Type 1 vs Type 2 — Comparison	p. 9
08	Virtual Machine Architecture	p. 10
09	VM Creation Process (Step-by-Step)	p. 11
10	Server Consolidation	p. 12
11	The Three Techniques of Virtualization	p. 13
12	Hardware-Assisted Virtualization	p. 14
13	Full vs Para Virtualization	p. 15
14	Containerization — An Overview	p. 16
15	VM vs Container — The Big Comparison	p. 17
16	Docker Architecture & Components	p. 18
17	Challenges of Virtualization	p. 19
18	Live Migration — Concept & Types	p. 20
19	Live Migration Process	p. 21
20	Key Exam Questions (2/5/10 marks)	p. 22
21	Sample Answers & Memory Tricks	p. 23
22	Quick Revision Cheat Sheet	p. 24

Introduction to Virtualization

DEFINITION

Virtualization is the process of creating a *virtual* (software-based) representation of something — such as a server, desktop, storage device, operating system, or network resources — rather than the actual physical version. A single physical machine can host many *virtual machines (VMs)*, each running its own OS and applications as if it were a separate physical computer.

Why it exists

Before virtualization, organizations ran **one application per physical server**. Studies showed that most servers used only **5–15%** of their capacity — the rest was wasted. Virtualization lets you split one big physical machine into many smaller, isolated virtual machines, dramatically improving utilization.

Key Idea — Decoupling

The fundamental idea of virtualization is to **decouple software from hardware**. Once decoupled, you can run any software on any hardware, move workloads between machines, and isolate failures.

```
# Traditional (1 server = 1 OS = 1 app)
+-----+ +-----+ +-----+
| Server 1 | | Server 2 | | Server 3 |
| App A   | | App B   | | App C   |
+-----+ +-----+ +-----+

# Virtualized (1 big server = many VMs)
+-----+
|          |
|  PHYSICAL  |
|  HARDWARE  |
|          |
|  VM 1      | |  VM 2      | |  VM 3      | |  VM 4      |
| App A     | | App B     | | App C     | | App D     |
+-----+
```

Short History

- **1960s** — IBM CP-40 and CP-67 mainframe introduced time-sharing and VM concepts.
- **1972** — IBM VM/370 made virtualization a commercial product for mainframes.
- **1998–1999** — VMware (founded 1998) brought virtualization to x86 servers.
- **2003** — VMware ESX Server (Type 1 hypervisor) launched.
- **2005+** — Intel VT-x and AMD-V added hardware virtualization extensions.
- **2013+** — Docker popularized *containerization* as a lightweight alternative.

Key Terms You Must Know

Host	the physical machine running virtualization	Guest	the VM that runs on top of the host
Hypervisor	software that creates and runs VMs (a.k.a. VMM)	VMM	Virtual Machine Monitor — same as hypervisor
VM	Virtual Machine — an isolated software "computer"	Workload	the application + OS running inside a VM

★ EXAM TIP

"Define virtualization in 2 lines" is a guaranteed question. Always mention *decoupling*, *multiple VMs on one physical machine*, and *better resource utilization*.

Benefits & Use Cases

Top 8 Benefits (Memorize This List!)

1. **Higher Resource Utilization** — Run many VMs on one server instead of leaving 85% of CPU idle.
2. **Cost Reduction** — Fewer physical machines → lower power, cooling, rack space, hardware cost.
3. **Easy Backup & Recovery** — A VM is just a file. Copy/snapshot the file → full system backup.
4. **Isolation** — One VM crashing doesn't affect others on the same host.
5. **Portability** — Move a VM from one physical server to another with zero downtime (live migration).
6. **Fast Provisioning** — A new VM spins up in minutes vs. days for a new physical server.
7. **Testing & Development** — Run multiple OS / browser versions on a single laptop.
8. **Disaster Recovery** — Replicate VMs to a second site, fail over in seconds.

Real-World Use Cases

Table 2.1 · Where virtualization is used in industry

Use Case	What it does
Server Consolidation	Replace many under-used servers with a few well-used virtualized servers.
Cloud Computing	AWS, Azure, GCP all run on top of virtualization — VMs are rented by the hour.
Software Testing	Developers run Windows, Linux, macOS, multiple browser versions, on one laptop.
Legacy App Support	Run an old Windows XP app inside a VM on modern hardware.
Disaster Recovery	Hot-standby VMs ready to take over if the primary fails.
Education / Labs	Each student gets an isolated VM for networking/OS lab practice.
Sandboxing	Run untrusted software inside a VM to keep the host safe.

MEMORY TRICK — "HIPPIES"

To remember the benefits in order: **H**igh utilization, **I**solation, **P**ortability, **P**rovisioning, **I**ncexpensive, **E**asy backup, **S**andboxing.

Before vs After — A Simple Example

```
# BEFORE: 10 separate physical servers, 10% used each
[Mail] [Web] [DB] [File] [App] [Test] [Dev] [QA] [Log] [Backup]
each server uses ~10% of its 16GB RAM → huge waste

# AFTER: 2 powerful servers, each running 5 VMs at ~70% utilization
+-- Server A (256GB RAM) --+      +-- Server B (256GB RAM) --+
| VM1 VM2 VM3 VM4 VM5 |          | VM6 VM7 VM8 VM9 VM10 |
+-----+                        +-----+
```

★ EXAM TIP

For "Discuss the benefits of virtualization" (10 marks), pick any **6 benefits**, write **2 lines** on each, and end with a real-world example. Always draws full marks.

Types of Virtualization

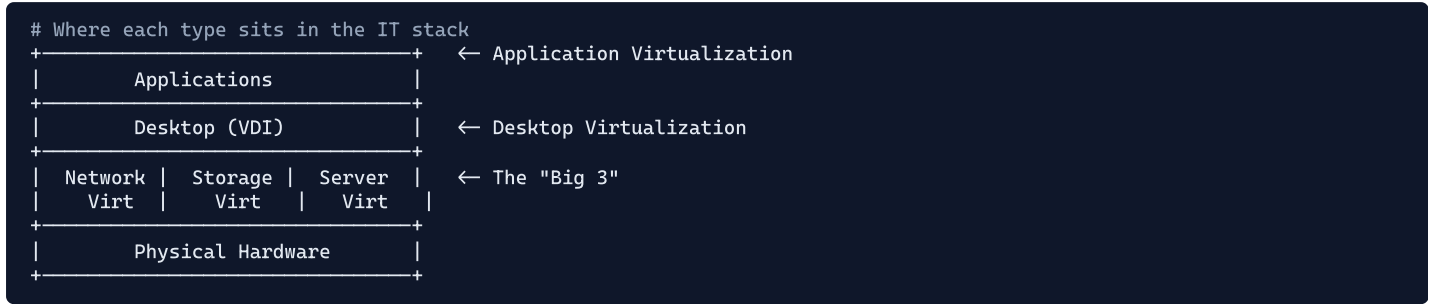
Virtualization is not just about servers. It can be applied to almost every layer of IT infrastructure.

Table 3.1 · The 5 main types of virtualization

Type	What is virtualized	Example / Use case
Server Virtualization	Physical server → many VMs, each with its own OS	VMware ESXi, Hyper-V — run 10 OSes on one box
Storage Virtualization	Many physical disks → one logical "pool" of storage	SAN, NAS, Amazon EBS, Google Cloud Storage
Network Virtualization	Physical network hardware → virtual switches, routers, VLANs	VMware NSX, Open vSwitch, Software-Defined Networking
Desktop Virtualization (VDI)	A user's desktop OS runs in a data-centre VM, streamed to a thin client	Citrix XenDesktop, VMware Horizon — "your desktop lives in the cloud"
Application Virtualization	An app runs in an isolated environment without installing on the OS	Docker containers, Java JVM, Wine, sandboxed browsers

Two more types you should know

- **Data Virtualization** — Abstract data from many sources into one logical view (used in analytics).
- **GPU Virtualization** — One physical GPU shared across many VMs (used in AI/ML clouds).



 **MEMORY TRICK — "SSNDA"**
 The 5 main types in order: **S**erver, **S**torage, **N**etwork, **D**esktop, **A**pplication. (Sounds like "S-S-N-D-A" — easy to recall.)

 **EXAM TIP**
 "List the types of virtualization" is a frequent 5-mark question. Always write a one-line definition + one real example for each. Examiners love examples.

Hypervisor (VMM) — The Heart of Virtualization

DEFINITION

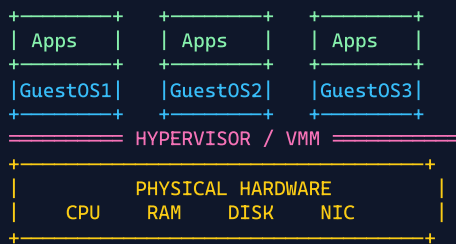
A **Hypervisor** (also called **VMM — Virtual Machine Monitor**) is a piece of software that creates, runs, and manages *virtual machines*. It sits between the hardware and the guest operating systems, and is responsible for sharing the physical resources (CPU, memory, disk, network) fairly among the VMs.

Two names, same thing

The terms **Hypervisor** and **VMM** are used interchangeably in textbooks. In exam answers, you can pick either — but the safer option is to write both: "Hypervisor (also called VMM)".

What does the hypervisor do?

1. **CPU Virtualization** — Shares CPU time across multiple VMs (scheduling + virtual CPU).
2. **Memory Virtualization** — Gives each VM its own private address space.
3. **I/O Virtualization** — Translates VM I/O requests to physical disk/network.
4. **Isolation** — One VM cannot read another VM's memory or steal its CPU.
5. **Snapshot & Migration** — Saves a VM's state, moves it to another host.



Three responsibilities summarized

Table 4.1 · The 3 core jobs of a hypervisor

Job	Meaning
1. Provisioning	Create, start, stop, delete VMs.
2. Resource Sharing	Divide CPU, RAM, disk, network among VMs.
3. Isolation	Keep one VM's failures / security breaches from spilling into another.

💡 MEMORY TRICK — "PSI"

The hypervisor's three jobs in order: **P**rovisioning, **S**haring resources, **I**solation.

★ EXAM TIP

Always draw the layered diagram (Apps → Guest OS → Hypervisor → Hardware) in your answer. Diagrams fetch easy marks even when your prose is average.

Type 1 Hypervisor (Bare-Metal)

DEFINITION

A **Type 1 (Bare-Metal / Native) Hypervisor** runs *directly on the physical hardware*, with no host operating system in between. The hypervisor itself acts as a specialized OS that boots first when the server powers on. All VMs then run on top of it.



Why "bare-metal"?

The hypervisor is installed on the "bare metal" — i.e. directly on the physical server with nothing else in between. It's the first software to load when the server powers on (after the BIOS/UEFI).

Famous Examples

Table 5.1 · Popular Type 1 hypervisors

Hypervisor	Vendor	Notes
VMware ESXi	VMware / Broadcom	Industry leader for enterprise data centres
Microsoft Hyper-V	Microsoft	Built into Windows Server
Xen / Citrix XenServer	Citrix / Linux Foundation	Powers AWS EC2 under the hood
KVM	Open Source (Linux kernel)	Turns Linux into a hypervisor; used in OpenStack
Oracle VM Server	Oracle	Based on Xen

Advantages

- ⚡ **Better performance** — direct access to hardware, no host OS overhead.
- 🔒 **Stronger isolation** — VMs cannot affect each other through a host OS.
- 📊 **Higher density** — can run more VMs per host (used in cloud data centres).

Disadvantages

- 🔧 Hardware drivers must be built into the hypervisor itself (limited driver support).
- 👤 Requires specialised admin skills to manage.
- 💰 Most enterprise products (ESXi, Hyper-V) are licensed.

★ EXAM TIP

"Type 1 hypervisor is also called bare-metal because it runs directly on the hardware without any host OS." Write this line — it earns you 1 free mark.

Type 2 Hypervisor (Hosted)

DEFINITION

A **Type 2 (Hosted) Hypervisor** runs as an *application on top of an existing host operating system* (like Windows, macOS or Linux). The host OS is responsible for the physical hardware; the hypervisor simply creates VMs as processes of the host.



Why "hosted"?

Because the hypervisor is **hosted** by (i.e. runs on top of) a normal operating system. The host OS handles all the device drivers, file systems, networking — the hypervisor just borrows these.

Famous Examples

Table 6.1 · Popular Type 2 hypervisors

Hypervisor	Vendor	Used for
Oracle VM VirtualBox	Oracle	Free, cross-platform — popular for students
VMware Workstation / Fusion	VMware / Broadcom	Pro desktop virtualization
Parallels Desktop	Parallels	Run Windows on macOS
QEMU	Open Source	Often paired with KVM on Linux

Advantages

- ✓ Easy to install — just install an app on your existing OS.
- ✓ Inherits all host drivers (USB, WiFi, printer, etc.).
- ✓ Great for development, learning, testing on a laptop.
- ✓ Most products are free (VirtualBox, QEMU).

Disadvantages

- 🐢 **Slower** — every VM I/O has to pass through the host OS, so there's an extra hop.
- 🔒 **Weaker isolation** — if the host OS crashes, all VMs crash with it.
- 📦 **Lower density** — not suitable for cloud-scale VM hosting.

MEMORY TRICK

Type 1 = 1 layer between VM and hardware (just the hypervisor). **Type 2** = 2 layers (hypervisor + host OS).

Type 1 vs Type 2 Hypervisor

This is one of the most-asked comparison questions in exams. Memorize this table.

Table 7.1 · Side-by-side comparison

Parameter	Type 1 (Bare-Metal)	Type 2 (Hosted)
Also called	Native, Bare-Metal	Hosted
Runs on	Directly on hardware	Inside a host OS
Host OS required?	No	Yes
Performance	High	Lower
Isolation	Strong	Weaker
VM density (per host)	High — hundreds possible	Low — tens at most
Hardware driver support	Limited (drivers in hypervisor)	Excellent (inherits host drivers)
Ease of install	Complex	Very easy
Used for	Enterprise data centres, cloud	Personal use, dev, testing
Examples	VMware ESXi, Hyper-V, Xen, KVM	VirtualBox, VMware Workstation, Parallels

At a Glance — Two Diagrams

Type 1 (Bare-Metal)

```
[ App | App ]
[GuestOS|GuestOS]
[ HYPERVISOR ]
[ HARDWARE ]
```

3 layers above hardware. Direct, fast.

Type 2 (Hosted)

```
[ App | App ]
[GuestOS|GuestOS]
[ HYPERVISOR ]
[ HOST OS ]
[ HARDWARE ]
```

4 layers above hardware. Slower but easy.

Which one is used in the cloud?

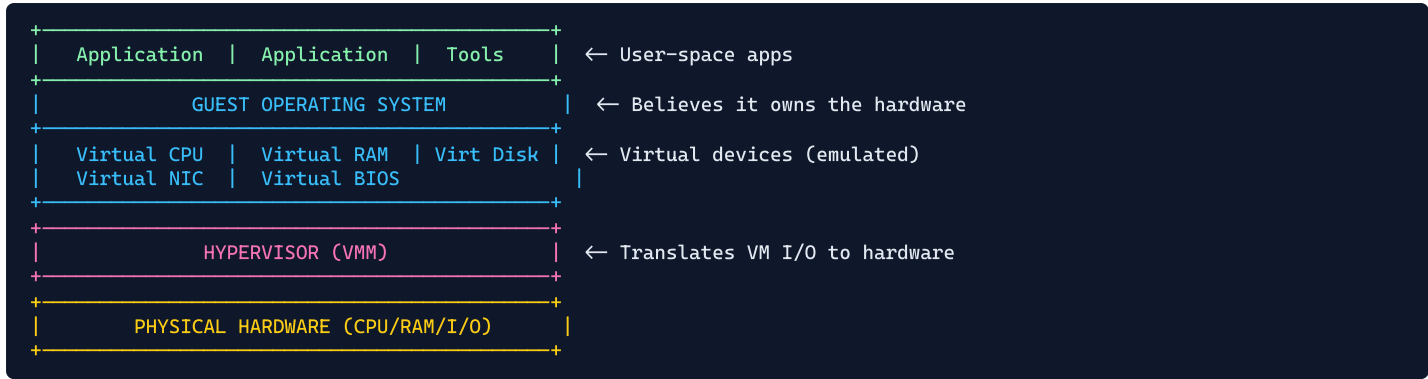
Public clouds (AWS, Azure, GCP) use **Type 1** hypervisors almost exclusively — they need the performance and density. AWS famously used a heavily-modified Xen; Azure uses Hyper-V; GCP uses KVM-based hypervisors.

★ EXAM TIP

"Differentiate between Type 1 and Type 2 hypervisor" — Always draw **both diagrams** side-by-side. Even a 2-line answer with a diagram gets full marks.

Virtual Machine Architecture

What a VM looks like, top to bottom



Components explained

Table 8.1 · Components of a Virtual Machine

Component	What it does
Virtual CPU (vCPU)	A virtual processor the guest OS sees. The hypervisor maps vCPUs to physical CPU cores.
Virtual RAM	Logical address space given to the VM; backed by physical RAM pages managed by the hypervisor.
Virtual Disk (VMDK / VHD / qcow2)	A file on the host's disk that the VM treats as its hard drive.
Virtual NIC (vNIC)	A virtual network card connected to a virtual switch on the host.
Virtual BIOS / UEFI	Boots the guest OS from the virtual disk, just like real firmware.
Virtual Devices	Emulated keyboard, mouse, USB, GPU — all software-defined.

VM Files — What's actually on disk?

- **.vmdk / .vhd / .qcow2** — virtual disk image
- **.vmx / .xml** — VM configuration (CPU count, RAM, network)
- **.nvram** — virtual BIOS settings
- **.log** — VM activity log
- **snapshot files** — saved states (memory + disk)

KEY INSIGHT

Because a VM is just *files* on a disk, you can **copy, move, back up, or even email** a VM. This is the magic of virtualization — the entire computer becomes a portable file.

VM Creation Process (Step-by-Step)

Creating a VM is the practical skill every exam wants you to understand. Here is the 7-step process (using VMware as the example — works similarly on Hyper-V, VirtualBox, KVM).

The 7 Steps

- 1. **Install the Hypervisor**
Boot the server, install ESXi / Hyper-V / KVM on the bare metal. (Type 1) or install VirtualBox on your laptop (Type 2).
- 2. **Configure Host Resources**
CPU virtualization (VT-x / AMD-V) enabled in BIOS. Network and storage connected.
- 3. **Open the Management Console**
vSphere Client, Hyper-V Manager, or VirtualBox UI. Connect to the hypervisor.
- 4. **Click "Create New VM"**
Choose a name, target host/folder, and resource pool.
- 5. **Allocate Hardware**
Pick how many vCPUs, how much RAM, and disk size for the VM. Attach a virtual NIC.
- 6. **Attach Installation Media**
Mount an ISO of the guest OS (e.g. `ubuntu-22.04.iso`) to the VM's virtual CD drive.
- 7. **Power On & Install OS**
Boot the VM → it sees the ISO as its boot disk → install the OS inside the VM → install VMware Tools / guest additions → done.

```
# Typical VM creation parameters
+-----+-----+
| VM Name      | Web-Server-01 |
| Guest OS     | Ubuntu 22.04 LTS |
| vCPUs        | 2              |
| RAM          | 4 GB           |
| Virtual Disk | 40 GB (thin)   |
| Network      | VM Network (NAT) |
| ISO mounted  | ubuntu-22.04.iso |
+-----+-----+
```

VM States (Lifecycle)

Table 9.1 · Power states of a VM

State	Meaning
Powered Off	VM is shut down, not consuming resources. Safe to delete.
Powered On	VM is running; OS is booted.
Suspended / Paused	VM state saved to disk; can be resumed instantly.
Snapshot	A point-in-time copy of disk + memory. Used before risky changes.
Cloned	An exact duplicate of the VM, with a new identity (MAC, hostname).

★ EXAM TIP

"Explain the steps of VM creation" — Always mention **VT-x/AMD-V must be enabled in BIOS**. Most students forget this and lose a mark.

Server Consolidation

DEFINITION

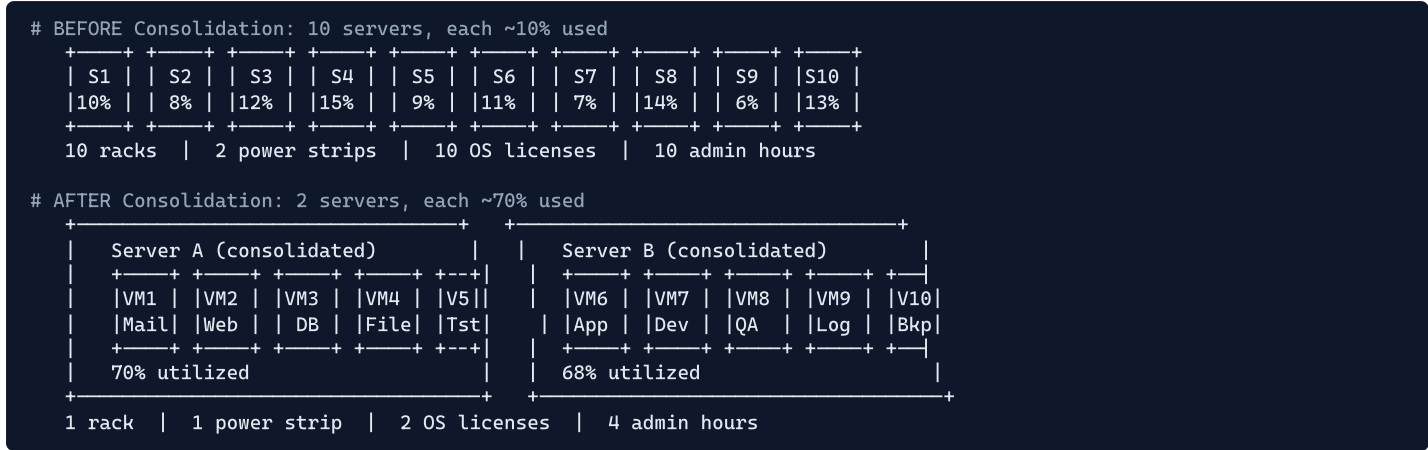
Server Consolidation is the practice of replacing many under-utilized physical servers with a few highly-utilized virtualized servers, by running each workload inside its own VM on the consolidated hardware.

The Problem

Studies show that the average enterprise server runs at only **5–15%** of its capacity. Buying one server per app is the traditional approach — but it leads to:

- Wasted CPU, RAM, power, cooling, rack space
- More hardware failures (more machines = more chances to fail)
- Higher licensing, admin, and electricity costs

Before & After — Visual



Types of Consolidation

Table 10.1 · Two flavours of consolidation

Type	How it works	Risk
Physical-to-Virtual (P2V)	Convert an existing physical server into a VM using a tool (VMware Converter, Disk2VHD).	Low — no change to applications
Virtual-to-Virtual (V2V)	Convert a VM from one hypervisor format (e.g. VMware) to another (e.g. Hyper-V).	Low — VMs are portable

Real Number Example (remember this!)

WORKED EXAMPLE

A company has **20 servers at 10% utilization**. They consolidate to **2 servers at 80% utilization**. Result: **90% less power, 90% less rack space, 90% fewer hardware failures** — and the same number of running apps.

The Three Techniques of Virtualization

DEFINITION

There are **three main techniques** by which a hypervisor can present hardware to a VM. Each has trade-offs between **performance**, **compatibility**, and **complexity**.

Technique 1 — Emulation

The hypervisor **interprets every instruction** of the guest. No code is run directly on the physical CPU. **Slowest** but most flexible — can emulate a different CPU architecture entirely (e.g. run ARM code on an x86 host).

```
# Emulation flow
Guest code (e.g. ARM) → Emulator reads each instruction → Translates to x86 → Runs on CPU
# Use cases: QEMU, Microsoft phone emulators, retro game emulators
```

Technique 2 — Full Virtualization

The guest OS is **unaware it is virtualized**. The hypervisor traps sensitive instructions and translates them on the fly (using binary translation or hardware extensions). The guest runs **native** instructions on the physical CPU for the most part — **fast**.

```
# Full Virt flow
Guest code → Mostly runs directly on CPU → Privileged calls intercepted → Translated by hypervisor
# Use cases: VMware ESXi, Hyper-V, KVM (with VT-x/AMD-V)
```

Technique 3 — Para-Virtualization

The guest OS is **modified** to know it is virtualized. It makes explicit "hypercalls" to the hypervisor instead of doing privileged operations. **Faster** than full virt but requires modifying the guest OS kernel.

```
# Para-virt flow
Modified guest → Replaces sensitive instructions with hypercalls → Hypervisor responds directly
# Use cases: Xen (original mode), VMware guest tools (paravirt drivers)
```

Side-by-Side Comparison

Table 11.1 · Three techniques compared

Parameter	Emulation	Full Virt	Para-Virt
Speed	Slowest	Fast	Fastest
Guest OS modified?	No	No	Yes
CPU architecture	Can be different	Must match	Must match
Hardware support	Not needed	VT-x / AMD-V helps	Not needed
Examples	QEMU, Bochs	VMware, Hyper-V, KVM	Xen, VirtIO

MEMORY TRICK — "EFP" = "EASY, FAST, PRO"

Order of speed: **Emulation** (slowest) < **Full Virt** < **Para-Virt** (fastest).
 Order of complexity: **Emulation** (most flexible) > **Full Virt** > **Para-Virt** (least flexible — needs modified guest).

Hardware-Assisted Virtualization

In the early 2000s, Intel and AMD added special CPU features to make full virtualization much faster. These features are critical to modern cloud computing.

The Problem They Solved

Before 2005, x86 CPUs had **17 "sensitive" instructions** that could not be safely virtualized by software alone. The hypervisor had to use slow **binary translation** for each one. Performance suffered.

Intel VT-x & AMD-V

Table 12.1 · Hardware virtualization extensions

Feature	Intel	AMD
Base CPU virtualization	VT-x (Intel Virtualization Technology)	AMD-V (SVM / Pacifica)
I/O virtualization	VT-d	AMD-Vi (IOMMU)
Network virtualization	VT-c	—
Year introduced	2005 (Pentium 4)	2006 (Athlon 64)

What VT-x / AMD-V actually does

1. Adds a new **"non-root" CPU mode** for the guest — sensitive instructions no longer crash, they "trap" to the hypervisor cleanly.
2. Adds **VMCS** (Virtual Machine Control Structure) — a special CPU data structure where the hypervisor stores guest state between traps.
3. Reduces the number of traps from thousands per second to a few.
4. Makes full virtualization performance close to native.

```
# Without hardware support (pre-2005)
Guest: "Read disk" → Hypervisor: translate (slow) → Hardware
~ 1000s of traps per second

# With VT-x / AMD-V (modern CPUs)
Guest: "Read disk" → CPU traps to hypervisor → Hypervisor handles → Hardware
~ 10s of traps per second
```

How to check if your CPU supports it

```
# Linux
$ grep -E 'vmx|svm' /proc/cpuinfo
# If 'vmx' appears → Intel VT-x supported
# If 'svm' appears → AMD-V supported

# Windows
Task Manager → Performance tab → CPU → "Virtualization: Enabled"
```

★ EXAM TIP

If asked "Name the hardware virtualization extensions", always write the full forms once: **VT-x (Intel Virtualization Technology)** and **AMD-V (Secure Virtual Machine)**. The examiner wants to see you know the abbreviations.

Full vs Para-Virtualization

Table 13.1 · Full vs Para-Virtualization

Parameter	Full Virtualization	Para-Virtualization
Guest OS modification	Not required	Required (kernel modified)
Performance	Near-native (with VT-x/AMD-V)	Near-native or better
Compatibility	Runs any OS, including Windows	Only open-source OSes (Linux, BSD)
Hardware support needed	Recommended (VT-x/AMD-V)	Not strictly required
Hypercall interface	None	Yes (explicit calls to hypervisor)
Implementation	Binary translation / HW assist	Modified kernel + hypercalls
Examples	VMware ESXi, Microsoft Hyper-V, KVM	Xen (PV mode), IBM z/VM, Xen with VirtIO

When to use which?

Use Full Virt when...

- You need to run Windows guests
- You want zero OS modification
- You have modern CPUs with VT-x/AMD-V
- You use proprietary apps that can't be re-compiled

Use Para-Virt when...

- You only run Linux/BSD
- You need the absolute lowest overhead
- You can modify the guest kernel
- You want predictable performance in HPC

Hybrid Approach (today's reality)

Most modern hypervisors use a **hybrid**: the CPU is virtualized with hardware assist (full virt) but I/O uses para-virtualized drivers (like **VirtIO** in KVM/QEMU, **VMCI** in VMware, **Hyper-V Integration Services**). This gives full OS compatibility with near-para-virt I/O performance.

🔑 QUICK MNEMONIC

Full Virt = "Full OS compatibility" (but slower without HW).

Para-Virt = "Par-tner with the hypervisor" (guest OS cooperates, runs faster).

Containerization — An Overview

DEFINITION

Containerization is a lightweight form of OS-level virtualization in which an application, its dependencies, libraries, and configuration are packaged into a single portable unit called a **container**. Containers share the host OS kernel but run in isolated user-space environments.

Why containers became popular

VMs are great for isolation, but they come with a heavy cost — each VM runs its own full OS, taking 1–10 GB of disk and 1–4 GB of RAM. For microservices (where you might run 100s of small apps), this is too much overhead.

Containers solve this: they share the host kernel, so they start in **milliseconds** and use only **tens of MB** of RAM. The trade-off is weaker isolation (one shared kernel = one shared attack surface).

Key Container Concepts

Table 14.1 · Container vocabulary

Term	Meaning
Image	A read-only template (like a snapshot) used to create containers. Built from a Dockerfile.
Container	A running instance of an image. Lightweight, isolated, ephemeral.
Dockerfile	A text file with instructions to build an image (FROM, RUN, COPY, CMD...).
Docker Hub	A public registry of pre-built images (like an app store for containers).
Docker Engine	The runtime that builds and runs containers on a host.
Orchestrator	Tool that manages many containers across many hosts (Kubernetes, Docker Swarm).



Why "containers" are called containers

Like a shipping container that holds any cargo, an OS container holds any application + its dependencies in a standardized way — so it runs the same on a developer's laptop, a test server, or a production cloud.

ONE-LINER TO REMEMBER

"VMs virtualize the **hardware**; containers virtualize the **OS**."

VM vs Container

This is the single most-asked comparison in any modern virtualization exam. Memorize this table.

Table 15.1 · Virtual Machines vs Containers

Parameter	Virtual Machine	Container
What is virtualized	Hardware	Operating System (kernel shared)
Has its own OS?	Yes — full guest OS	No — shares host kernel
Boot time	30 s – 2 min	Milliseconds – 1 s
Disk size	10 – 100 GB per VM	10 – 500 MB per image
RAM footprint	1 – 32 GB per VM	10 – 500 MB per container
Density (per host)	10s – 100s	1000s
Isolation	Strong (separate kernel)	Weaker (shared kernel)
Security	Higher	Lower (kernel-level escapes possible)
Performance	~95–98% of native	~99% of native
Can run different OS?	Yes (Linux on Windows, etc.)	No — must match host kernel
Examples	VMware ESXi, Hyper-V, KVM	Docker, containerd, Podman, LXC

Visual Comparison

VM Stack (heavy)

```
[App | App ]
[Bins/Libs  ]
[Guest OS - GBs]
[HYPERVISOR ]
[HARDWARE   ]
```

Each VM has its own kernel.

Container Stack (light)

```
[App|App|App|App]
[Bins/Libs (small)]
[DOCKER ENGINE  ]
[HOST OS KERNEL ]
[HARDWARE       ]
```

All containers share the host kernel.

When to use which?

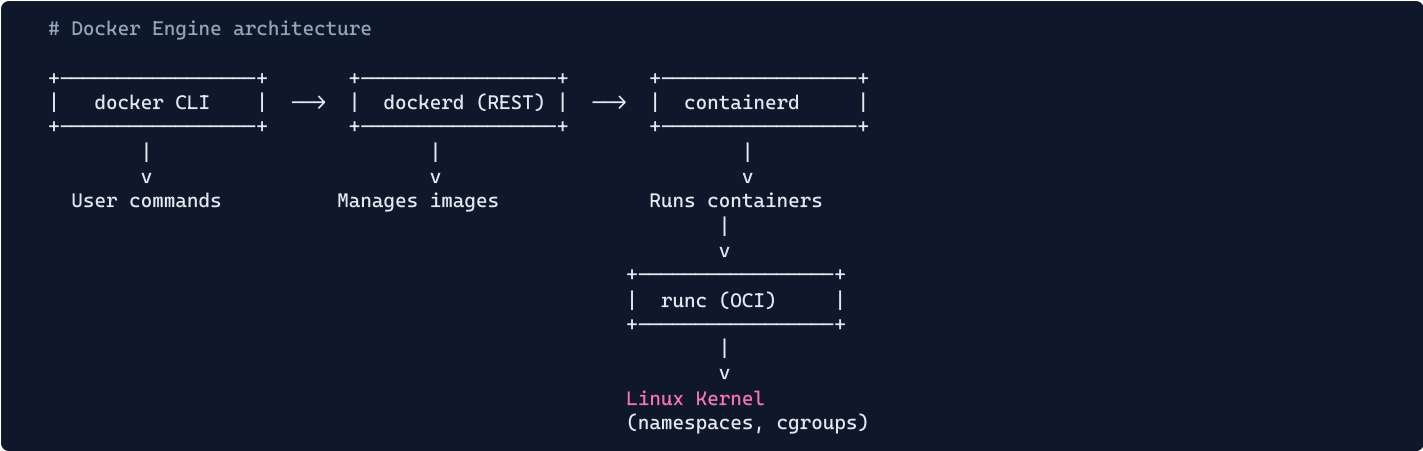
- 👉 **Use VMs when** you need strong isolation, different OSes, or you're running monolithic legacy apps.
- 👉 **Use Containers when** you're running microservices, want fast deployment, or need high density on one host.
- 👉 **Use BOTH** — many production systems run containers *inside* VMs (e.g. Kubernetes on AWS EC2).

★ EXAM TIP

For "Compare VMs and Containers" (10 marks): draw both stacks side-by-side, then write a 10-row table (above), then end with one line — "in practice, the two are often used together".

Docker Architecture & Components

Docker in One Picture



Docker Components

Table 16.1 · Core Docker components

Component	Role
Docker Client (CLI)	The <code>docker</code> command you type in a terminal.
Docker Daemon (dockerd)	Background service that manages images, containers, networks, volumes.
Docker Images	Read-only templates (built from Dockerfile) used to create containers.
Docker Containers	Running instances of images — the actual "place" where apps run.
Docker Registry	Stores images. Docker Hub is the public default; private registries are common in companies.
Dockerfile	Recipe to build an image. Each line is a layer.
Docker Compose	A YAML file to define and run multi-container apps (e.g. web + database).
Docker Volumes	Persistent storage that survives container restarts.

How a Container Actually Works (Linux Magic)

Linux provides two kernel features that make containers possible:

- **Namespaces** — give each container its own view of the system (its own PID space, its own network stack, its own filesystem mount). *This is what provides isolation.*
- **cgroups** (control groups) — limit how much CPU, RAM, disk I/O a container can use. *This is what provides resource control.*

```
# Kernel features that make containers possible
Linux Kernel

+-----+
| Namespaces | cgroups | UnionFS | Networking |
| (isolation) | (limits)  | (layers) | (veth, NAT) |
+-----+

Namespaces give each container its own:
PID  NET  MNT  UTS  IPC  USER
(process)(network)(filesystem)(hostname)(IPC)(users)
```

🧠 QUICK MNEMONIC

"Docker is a packaging format" — the image is the package, the container is the running app, the registry is the warehouse.

Challenges of Virtualization

Virtualization is not a silver bullet. It introduces new problems that every architect must consider.

Table 17.1 · Major challenges of virtualization

Challenge	What goes wrong	How to mitigate
Performance overhead	Hypervisor adds a small CPU/RAM cost. I/O heavy workloads suffer most.	Use Type 1 hypervisors + VT-x/AMD-V + para-virt drivers.
Security	If the hypervisor is compromised, all VMs are compromised. VM escape attacks exist.	Patch hypervisor; minimize hypervisor attack surface; use signed images.
Single point of failure	One host down = many VMs down at once.	Live migration to a healthy host; clustered hosts (HA, FT).
Resource contention	"Noisy neighbour" — one VM hogs CPU/RAM, others slow down.	Resource pools, quotas, monitoring, alerts.
Licensing complexity	Microsoft, Oracle license per VM, not per host — costs can balloon.	Use per-core licensing, open-source stacks, or Linux guests.
Storage I/O bottleneck	Many VMs sharing one disk array causes latency.	Use SSDs, NVMe, SAN, or distributed storage (vSAN, Ceph).
Backup & recovery	Backing up a running VM is tricky (consistent snapshots need quiescing).	Use snapshot tools, application-aware backups, replication.
Management complexity	Tens to thousands of VMs is hard to track manually.	Use management tools (vCenter, System Center, OpenStack).

The "VM Sprawl" Problem

Because VMs are so easy to create, organizations often end up with **VM sprawl** — hundreds of zombie VMs nobody knows about, consuming resources. The fix is a **VM lifecycle policy**: require justification for every new VM, automatically retire idle ones.

Security Concerns in Detail

- **VM Escape** — A bug lets code inside a VM break out and run on the host. Famous example: 2017 Cloudbleed in Xen.
- **Hypervisor rootkit** — Malware that hides inside the hypervisor (Blue Pill attack, 2006).
- **Data leakage** — Shared hardware caches (CPU) can leak data between VMs (Spectre / Meltdown, 2018).

★ EXAM TIP

"Discuss the challenges of virtualization" — Pick any **4 challenges** from the table and write **3 lines each**: what the problem is, why it happens, and how to fix it. This is a 10-mark answer pattern.

Live Migration — Concept & Types

DEFINITION

Live migration is the process of moving a running virtual machine from one physical host (the *source*) to another (the *destination*) **without stopping the workload**. From the user's perspective, the application keeps running throughout the move — usually with a brief pause of **< 1 second**.

Why Live Migration matters

-  **Server maintenance** — patch the host OS or replace hardware with zero downtime.
-  **Load balancing** — move a busy VM to a less-busy host automatically.
-  **Power management** — consolidate VMs onto fewer hosts at night, shut down idle hosts to save power.
-  **Disaster avoidance** — evacuate VMs from a host that's about to fail (overheating, disk error).

Three Types of Live Migration

Table 18.1 · Types of live migration

Type	How it works	Pros / Cons
Pre-Copy	Memory is iteratively copied while the VM keeps running. Stop-and-copy at the end.	+ Simple, widely used (Xen, KVM, VMware). - Slow for write-heavy VMs.
Post-Copy	VM is suspended first; minimal state sent; memory is pulled on demand by the destination.	+ Short downtime, less data sent. - Network round-trips for every page fault.
Hybrid	Starts as pre-copy; if VM is too write-heavy, falls back to post-copy for those pages.	+ Best of both worlds. - Complex to implement.

What gets migrated?

```
# State that must be moved to the new host
```

SOURCE HOST (old)		DESTINATION HOST (new)
vCPU state	→	vCPU state
Memory (RAM pages)	→	Memory (RAM pages)
Disk (VMDK / qcow2)	→	Disk (shared over SAN)
Network connections	→	Network connections
Open files & sockets	→	Open files & sockets
Device state	→	Device state

MNEMONIC — "PPP"

Three types in order: **P**re-copy, **P**ost-copy, **P**ybrid. Remember "Pre-Post-Puzzle" — pre-copy is the most common, post-copy is the alternative, hybrid is the puzzle of both.

Live Migration Process (Pre-Copy)

Pre-copy is the most common live-migration technique. Here is its 6-step algorithm, used by Xen, KVM, and VMware vMotion.

Step-by-Step

1. Pre-Migration

Destination host is selected. Both hosts reserve the resources the VM needs (CPU, RAM, network).

2. Reservation

A container (VM skeleton) is created on the destination with no state yet — just reserved capacity.

3. Iterative Pre-Copy

Memory is copied to the destination in rounds. While copying, the VM is still running on the source — pages that change during the copy are marked *dirty* and will be re-copied in the next round.

4. Stop-and-Copy

When the remaining dirty pages fall below a threshold (e.g. < 50 MB), the VM is briefly paused on the source. The remaining dirty pages and final CPU/device state are sent to the destination.

5. Commitment

The destination VM is activated. The source VM is told it is no longer the primary.

6. Activation

The destination resumes the VM. Network traffic is redirected (often via gratuitous ARP). The VM continues running — users see a sub-second pause, if anything.

```
# Pre-copy rounds - memory shrinks each round
Round 1: |##### dirty = 4 GB      | copy 4 GB
Round 2: |##### dirty = 2 GB      | copy 2 GB
Round 3: |### dirty = 800 MB        | copy 800 MB
Round 4: |# dirty = 200 MB          | copy 200 MB
Round 5: | dirty = 40 MB (below threshold → STOP) | copy 40 MB + CPU state

Total data sent ~ 7 GB | Downtime ~ 200 ms
```

The Dirty Page Problem

What if a VM is so write-heavy that new dirty pages arrive as fast as you can copy them? This is called **convergence failure**.

Solutions:

- Cap the number of pre-copy rounds (e.g. 5 rounds max).
- Force a stop-and-copy if rounds exceed the cap.
- Switch to post-copy or hybrid in extreme cases.

Networking Trick — Gratuitous ARP

When the VM "moves" to the destination, the destination sends a **gratuitous ARP** on the network saying "the VM with IP X is now at MAC Y" (the destination's MAC). Switches update their tables, and traffic is seamlessly redirected. The user never knows the VM moved.

Requirements for Live Migration

Table 19.1 · What the infrastructure must provide

Requirement	Why
Shared Storage (SAN / NFS)	The VM's virtual disk must be accessible by both source and destination.
Same CPU family on both hosts	VT-x instructions must match — Intel-only or AMD-only clusters.
Fast network (1 Gbps+)	Memory pages are copied over the network.
Compatible hypervisor version	Source and destination must run the same (or compatible) hypervisor.

★ EXAM TIP

For "Explain live migration", always list the 6 steps above and include the **iterative pre-copy diagram**. Mention **shared storage** as a prerequisite — most students forget this.

Key Exam Questions (2/5/10 marks)

2-Mark Questions (Short — 2–3 lines each)

Q1. Define virtualization. [2 marks]
Q2. What is a hypervisor? [2 marks]
Q3. Differentiate between Type 1 and Type 2 hypervisor (any 2 points). [2 marks]
Q4. What is server consolidation? [2 marks]
Q5. Name the three techniques of virtualization. [2 marks]
Q6. What is VT-x? [2 marks]
Q7. Define containerization. [2 marks]
Q8. What is live migration? [2 marks]
Q9. Name the two hardware-virtualization extensions for x86 CPUs. [2 marks]
Q10. What is the difference between a VM and a container? [2 marks]

5-Mark Questions (Medium — diagrams expected)

Q11. Explain the architecture of a Type 1 hypervisor with a neat diagram. [5 marks]
Q12. Compare full virtualization and para-virtualization. [5 marks]
Q13. Explain server consolidation with a before-after example. [5 marks]
Q14. List and explain any 5 types of virtualization. [5 marks]
Q15. With a diagram, explain the VM creation process. [5 marks]

10-Mark Questions (Long — full answers, multiple diagrams)

Q16. What is a hypervisor? Explain Type 1 and Type 2 hypervisors in detail with diagrams. [10 marks]
Q17. Discuss the three techniques of virtualization. How does hardware-assisted virtualization improve performance? [10 marks]
Q18. Compare virtual machines and containers. Discuss Docker architecture in detail. [10 marks]
Q19. Explain the concept of live migration. Discuss pre-copy, post-copy, and hybrid techniques with diagrams. [10 marks]
Q20. What is virtualization? Discuss its benefits, types, and challenges in modern data centres. [10 marks]

Sample Answers & Memory Tricks

Model Answer — "Define Virtualization" (2 marks)

Answer:

Virtualization is a technology that allows a single physical machine (the host) to run multiple isolated virtual machines (guests), each with its own operating system and applications, by decoupling software from the underlying hardware.

Example: VMware ESXi running 10 Linux VMs on one physical server.

Model Answer — "Differentiate Type 1 and Type 2" (5 marks)

Answer:

Type 1 (Bare-Metal) Hypervisor runs directly on the physical hardware without any host OS. Examples: VMware ESXi, Microsoft Hyper-V, Xen, KVM. Used in enterprise data centres and cloud.

Type 2 (Hosted) Hypervisor runs as an application on top of a host operating system. Examples: Oracle VirtualBox, VMware Workstation, Parallels. Used for personal, development and testing purposes.

Differences:

1. Type 1 has no host OS; Type 2 requires a host OS.
2. Type 1 gives better performance and isolation; Type 2 is slower but easier to install.
3. Type 1 supports higher VM density; Type 2 supports fewer VMs per host.

Model Answer — "Live Migration" (10 marks — short version)

Answer:

Live migration is the movement of a running VM from one physical host to another without stopping the workload. It is used for load balancing, hardware maintenance, and disaster avoidance.

Three techniques:

1. **Pre-copy** — memory is copied iteratively while the VM runs; final stop-and-copy sends remaining state.
2. **Post-copy** — VM is suspended, minimal state is sent; remaining memory pages are fetched on demand.
3. **Hybrid** — combines both; switches if pre-copy can't converge.

Prerequisites: shared storage, same CPU family, fast network, compatible hypervisor.

Example: VMware vMotion, used in vSphere to move running VMs between ESXi hosts with < 1 s downtime.

Top 5 Memory Tricks (cheat sheet)

Topic	Trick
Types of Virt	SSNDA — Server, Storage, Network, Desktop, Application
Hypervisor jobs	PSI — Provisioning, Sharing, Isolation
Benefits of Virt	HIPPIES — High util, Isolation, Portability, Provisioning, Inexpensive, Easy backup, Sandboxing
Techniques speed	EFP — Emulation < Full < Para (slowest to fastest)
Live migration types	PPP — Pre, Post, Hybrid

Quick Revision Cheat Sheet

Definition Pocketbook

Virtualization decouple s/w from h/w; run many VMs on one server

Type 1 bare-metal, runs on h/w, no host OS

Emulation slow, can change CPU arch

Para-Virt modified guest, hypercalls

AMD-V AMD HW virt

Docker most popular container engine

Hypervisor / VMM software that creates & runs VMs

Type 2 hosted, runs inside a host OS

Full Virt unmodified guest, fast with HW

VT-x Intel HW virt

Container OS-level virt, shares kernel

Live migration move running VM with <1 s downtime

One-Line Facts (write these last)

- Virtualization = decoupling **software from hardware**.
- Type 1 hypervisor = **no host OS**; Type 2 = **needs host OS**.
- Three techniques = **Emulation, Full Virt, Para Virt**.
- Para-virt needs **modified guest OS**; full virt doesn't.
- VT-x (Intel) and AMD-V (AMD) enable **hardware-assisted** virtualization.
- VMs virtualize **hardware**; containers virtualize the **OS**.
- Containers share the host **kernel**.
- Docker is built on **namespaces + cgroups**.
- Live migration needs **shared storage + fast network**.
- vMotion, Xen, KVM all use the **pre-copy** algorithm.

Numbers to Remember

Number	What it means
5–15%	Typical utilization of an un-virtualized server (the problem virtualization solves)
17	Number of x86 sensitive instructions that VT-x / AMD-V made safely virtualizable
2005 / 2006	Year Intel VT-x / AMD-V launched
< 1 second	Typical downtime of a live migration
1000s	Number of containers a single host can run (vs tens of VMs)

The 5-Point Summary (write this on your hand before the exam!)

1. Virtualization creates many VMs on one physical machine.
2. A hypervisor (Type 1 or Type 2) makes it possible.
3. Three techniques — emulation, full, para — balance speed vs compatibility.
4. Hardware assist (VT-x / AMD-V) makes full virtualization near-native.
5. Containers (Docker, Kubernetes) are the next step: lightweight, fast, OS-level.

FINAL TIP

Read the question **twice**. If it says "explain" — use a diagram + 3–4 lines. If it says "list" — bullet points only. If it says "compare" — table. Match your answer format to the verb. That's how you score full marks.